스마트창고 - 주문 배출 + 재고관리 시스템

폴더 구성

파일	보드/환경	역할
`ESP32.ino` + `config.h`	ESP32	총괄 두뇌. Mobius 통신, 주문 해석, 재고 관리, 우노에 배출 명령
`Uno.ino`	아두이노 우노 R4 WiFi	명령 받아서 실제 서보(스토퍼) 동작
`test_order.py`	PC (VS Code)	테스트용. 랜덤 주문 생성 + 재고부족 알림 + 카트리지 교체 요청

---

1. 각 코드가 하는 일

ESP32.ino (총괄)

전체 시스템의 두뇌. 세 가지 일을 1초 주기로 반복함.

① 새 주문 감지 (cnt_order 폴링)

- Mobius의 `cnt_order/la`(최신 데이터)를 1초마다 조회
- 응답에 들어있는 `ri`(고유 번호)가 직전과 다르면 → "새 주문이 들어왔다"고 판단
- 주문 JSON(`keycap`, `colors`, `switch`, `board`)을 파싱

② 레일 매핑 → 배출 명령

- 주문을 "어느 층, 어느 레일"로 변환
  - 색상 → 층: r=0(1층), y=1(2층), b=2(3층), g=3(4층)
  - 글자 → 레일: E=0, I=1, S=2, N=3, F=4, T=5, P=6, J=7, 축=8
- 우노에 HTTP 요청 전송 (`/dispense?floor=0&rail=3&count=1`)
- **한 주문당 총 5번 전송**: 키캡 4개(개별) + 축 1번(4개 묶음으로 한 번에 배출)

③ 재고 관리

- 배출 성공(우노가 200 응답)했을 때만 재고 차감
- 재고 5개 이하면 경고 로그, 0이면 배출 자체를 스킵
- 재고가 바뀔 때마다 Mobius `cnt_stock`에 최신 재고 전체를 업로드
- `cnt_replace`도 함께 폴링해서, 교체 요청이 오면 카트리지 리필 실행

Uno.ino (층별 실행)

- WiFi에 붙어서 간이 웹서버를 띄움
- ESP32가 보낸 `/dispense?...` 요청을 받으면 이중 스토퍼 시퀀스 실행
  - STEP1: 뒤 스토퍼 닫힘 (키캡 1개 분리)
  - STEP2: 앞 스토퍼 열림 (통과)
  - STEP3: 앞 스토퍼 닫힘
  - STEP4: 뒤 스토퍼 열림 (다음 준비)

- `count` 값(1 또는 4)은 로그로만 확인하고, **스토퍼 동작은 항상 1회**
  (축은 물리적으로 한 번 열릴 때 4개가 동시에 나가는 구조라서)

test_order.py (PC 테스트 도구)

관리자 화면이 아직 없어서, 그 역할을 대신하는 테스트 도구. 명령어 3가지로 동작함.

입력	동작
Enter	랜덤 주문 1건을 `cnt_order`에 업로드 (MBTI 4글자 + 색상 랜덤 + 축 랜덤)
`floor,rail` (예: `1,0`)	해당 레일의 `cnt_replace` 교체 요청 업로드
q	종료

주문 업로드 후 동작

- 주문을 올린 뒤 그냥 끝나는 게 아니라, **ESP32가 실제로 처리해서 `cnt_stock`을 갱신할 때까지 기다림**
- `cnt_stock` 최신 CIN의 RI가 바뀌면 "처리 완료"로 판단하고, 그 시점의 재고 부족 목록을 자동으로 출력
- 대기 중에는 점(`...`)이 찍히면서 진행 상태를 표시

재고 부족 알림

- 매번 입력 대기 상태로 돌아올 때마다 최신 재고를 조회해서, 5개 이하인 항목을 목록으로 보여줌
- 각 항목 옆에 그대로 복사해서 입력할 수 있는 형식(`0,3`)이 같이 표시됨

교체 요청 시 안전장치

- 재고가 5개 초과인 레일을 입력하면 → "교체 불필요" 메시지 뜨고 요청 안 보냄
- floor는 0~3, rail은 0~8 범위를 벗어나면 → 입력 오류 처리
- 교체 요청을 보낸 뒤에는 ESP32가 리필을 끝내고 재고를 갱신할 때까지 기다렸다가, 완료되면 갱신된 재고 수량을 출력

---

2. 카트리지 교체 메커니즘

레일 하나당 재고를 3가지로 나눠서 관리함.

구분	의미
`stock`	지금 카트리지에 로드되어 배출 가능한 재고 (용량 50)
`totalPool`	아직 안 꺼낸 예비 재고 (레일당 상한: 키캡 300개 / 축 500세트)
`spareBin`	교체할 때 남아있던 자투리(0~5개)를 모아두는 통

교체 요청이 오면:

1. 남아있던 자투리를 `spareBin`에 적립 (버리지 않음)
2. `spareBin`이 50 이상이면 → 그걸로 리필 (재활용)
3. 아니면 `totalPool`에서 50개 꺼내 리필
4. `totalPool`도 50 미만이면 → 있는 만큼만 부분 리필
5. 완전히 바닥나면 → 리필 불가 (실물 보급 필요)

교체는 **자동이 아님.** 오직 `cnt_replace`에 요청이 올라왔을 때만 실행됨.

---

3. 사용 방법 (실행 순서)

STEP 1. 아두이노 우노에 코드 업로드

- `Uno.ino` 열고 `ssid`, `password`를 사용할 WiFi(핫스팟) 정보로 수정
- 보드 선택: `Arduino UNO R4 WiFi`
- 업로드 후 **시리얼 모니터(9600)** 열어서 출력된 **IP 주소 확인 및 메모**

STEP 2. ESP32 config.h 수정

- `config.h` 탭의 `UNO_TEST_IP`에 방금 확인한 우노 IP 입력
- `WIFI_SSID`, `WIFI_PASSWORD`도 우노와 **같은 WiFi**로 맞추기
- Mobius 인증 정보(API키, 강의코드, 오리진, 식별코드)가 채워져 있는지 확인

STEP 3. ESP32에 코드 업로드

- 보드 선택: `ESP32 Dev Module`
- 업로드 후 **시리얼 모니터(115200)** 열기
- "WiFi 연결됨" + "초기 재고 설정 완료" + "폴링 시작" 로그 확인

STEP 4. VS Code에서 파이썬 실행

- `.env` 파일에 COSS 접속 정보가 들어있는지 확인
- `python test_order.py` 실행
- **Enter** 눌러서 랜덤 주문 생성 → ESP32 처리 완료까지 자동 대기 → 재고 현황 출력
- 재고가 5개 이하로 떨어진 항목이 뜨면 `1,0` 같은 형식으로 입력해서 교체

STEP 5. 작동 확인

- ESP32 시리얼 모니터에서 주문 감지 → 배출 → 재고 차감 로그 확인
- 우노 시리얼 모니터에서 서보 동작 로그 확인
- COSS(Mobius)에서 `cnt_stock` 값이 갱신되는지 확인

---

4. 확인해야 할 것 (체크리스트)

우노 쪽

- 시리얼 모니터에 IP가 정상 출력되는지 (`0.0.0.0`이면 WiFi 연결 실패)
- `/dispense` 요청 수신 로그가 뜨는지
- 서보 2개가 STEP1~4 순서대로 움직이는지
- 한 주문당 스토퍼 동작이 **5번** 반복되는지

ESP32 쪽

- "★ 새 주문 감지! ★" 로그가 뜨는지
- 주문 파싱 결과(키캡/색상/스위치)가 실제 주문과 일치하는지
- 우노 전송이 5번(키캡4 + 축1) 나가고 전부 "성공"인지

- 재고 차감 로그가 정확한지 (키캡은 1개씩, 축은 1세트씩)
- "cnt_stock 동기화 성공" 로그가 뜨는지
- 교체 요청 보냈을 때 "★ 교체 요청 감지! ★" → 리필 로그가 뜨는지

파이썬 쪽

- Enter 후 "재고 반영 대기" → "재고 반영 완료"로 넘어가는지 (안 넘어가면 ESP32가 멈춰있는 것)
- 재고 부족 목록에 뜬 `floor,rail` 값이 실제 품목과 맞는지
- 교체 요청 후 "교체 완료" 메시지와 함께 재고가 리필된 수량으로 표시되는지

COSS(Mobius) 쪽

- `cnt_stock/la` 조회 시 최신 재고가 반영되어 있는지
- `cnt_replace`에 교체 요청 CIN이 올라가는지

---

5. 자주 나오는 문제

증상	원인/해결
우노 IP가 `0.0.0.0`	WiFi 연결 실패. 핫스팟 껐다 켜고 리셋 버튼 누르기
ESP32에서 "실패, 코드: -1"	우노 IP가 바뀌었을 가능성. 우노 시리얼에서 IP 다시 확인 후 `config.h` 수정
404 응답	AE 이름이나 컨테이너 이름 오타. 대소문자 구분함 (`ae_sjuiot`)
파이썬이 "재고 반영 대기"에서 안 넘어감	ESP32가 주문을 처리 못 하는 중. ESP32 시리얼 모니터 확인 (WiFi 끊김, 우노 응답 없음 등)
서보가 떨리기만 하고 안 움직임	전류 부족. USB 대신 별도 전원 사용
테스트를 처음부터 다시 하고 싶음	보드의 **리셋 버튼**을 누르면 재고가 초기값으로 돌아감 (재업로드 불필요)

---

6. 현재 상태 / 남은 작업

검증 완료

- ESP32 ↔ 우노 무선 통신
- Mobius 인증 / 주문 폴링 / 파싱
- 레일 매핑 (색상→층, 글자→레일)
- 재고 차감 + cnt_stock 동기화
- 카트리지 교체 + 자투리 재활용

아직 안 된 것 (TODO)

- `triggerDispense()`가 층 구분 없이 무조건 `UNO_TEST_IP` 하나로만 보냄
  → 우노 4대로 늘어나면 `floor` 값에 따라 IP 분기 필요
- PCA9685(서보 드라이버) 제어 코드 미적용 (지금은 우노에 서보 직결)
- AGV 도착 신호 연동 / 배출구 개방 로직
- 배출 완료 신호를 Mobius에 올리는 부분
- WiFi 끊겼을 때 재연결 로직 (장시간 전시 대비)